



GitHub Trending Top 10

每日热门开源项目深度解读

2026-09-30

今日榜单

- 1 NVIDIA/OpenShell** Rust
OpenShell is the safe, private runtime for autonomous AI agents. 2天
- 2 debpalash/VoiceStudio** Python
VoiceStudio is the open-source, fully-local ElevenLabs alternative — voice cloni... 4天
- 3 mvswarz/openrig** TypeScript
Multi-agent harness that runs Claude Code and Codex together as one system 4天
- 4 mksglu/context-mode** TypeScript
Context window optimization for AI coding agents. Sandboxes tool output (98% red... NEW
- 5 DietrichGebert/ponytail** JavaScript
Makes your AI agent think like the laziest senior dev in the room. The best code... NEW
- 6 harry0703/MoneyPrinterTurbo** Python
利用 AI 大模型和自动化工作流，根据主题或关键词一键生成高清短视频。Generate HD short vi... NEW
- 7 openclaw/openclaw** TypeScript
The AI that really does things. Any OS. Any Platform. The lobster way. 🦞 NEW
- 8 ComposioHQ/awesome-claude-skills** Python
A curated list of awesome Claude Skills, resources, and tools for customizing Cl... NEW
- 9 mattpocock/skills** Shell
Skills for Real Engineers. Straight from my .agents directory. NEW
- 10 heygen-com/hyperframes** TypeScript
Write HTML. Render video. Built for agents. NEW

#1

Rust

连续 2 天

OpenShell

OpenShell is the safe, private runtime for autonomous AI agents.

Stars **11,828** Forks **1,497** Today **+1,280**

<https://github.com/NVIDIA/OpenShell>

项目简介：OpenShell 是 NVIDIA 推出的面向自主 AI Agent 的安全私有运行时。它要解决的核心矛盾是：Agent 越能读文件、装包、调 API、用凭证就越有用，但直接放开又意味着数据和密钥的失控。OpenShell 的做法是让 Agent 保留这些能力，同时通过策略声明把它的可触及范围严格限定住。

核心功能：

- **内核级隔离与强制执行：**每个 Agent 跑在独立沙箱里，文件访问、系统调用、网络连接都在内核层被策略拦截和检查，网络请求离开沙箱前必须过策略关。
- **凭证托管：**Agent 永远拿不到真实凭证，OpenShell 只在请求发往已批准端点时才注入凭证，从根上防止密钥泄露。
- **策略变更的形式化验证：**策略改动生效前，用形式化验证预判它会新开放哪些访问（比如用凭证访问新主机、调用新 API），有风险的改动需人工审核。
- **网关控制面与 K8s 集成：**通过 Gateway 统一管理沙箱、策略和访问，可用 Helm 部署到 Kubernetes（要求 CNI 支持 NetworkPolicy）。
- **可扩展性与 Agent 技能包：**提供 middleware、interceptor、compute driver 扩展点，并附带 skills/ 让编码 Agent 自己学会驱动 CLI 和写策略。

适用场景：适合需要让 AI Agent 在真实环境中执行操作、但又不能放任其接触生产数据和密钥的团队，比如企业内部的自动化运维、代码助手、数据分析 Agent。也适合在 Kubernetes 上批量运行 Agent 集群、需要统一策略治理的平台团队。

技术亮点：用 Rust 实现，把安全边界下沉到内核而非应用层，这是它区别于多数"Agent 框架"的关键。更少见的是引入形式化验证来做策略变更的"事前审计"——不是运行后报警，而是应用前证明会开放什么权限，这在 Agent 安全领域是相当硬核的思路。凭证按端点注入的设计也把"最小权限"落到了网络请求粒度。

上手建议：环境需 Linux、Apple Silicon 的 macOS 或 WSL 2，加 Docker/Podman。最快路径是跑官方安装脚本后 `openshell sandbox create --name demo`，再跟着"Run Your First Agent"教程用 OpenCode 配免费 OpenRouter 模型，体验 Agent 请求新权限时如何审批。想贡献可从 skills/、策略示例或扩展接口入手，先读架构文档理解 gateway、supervisor、sandbox 三者的关系。

#2

Python

连续 4 天

VoiceStudio

VoiceStudio is the open-source, fully-local ElevenLabs alternative — voice cloning, voice design, video dubbing, dictation, transcription & audiobook creation in 646 languages.

Stars **49,980** Forks **5,553** Today **+3,481**

<https://github.com/debpalash/VoiceStudio>

项目简介： VoiceStudio 是一个完全本地运行的开源语音 AI 工作台，定位为 ElevenLabs 的开源替代方案。它把语音克隆、语音设计、视频配音、听写、转录和有声书制作整合到一个桌面应用里，支持 646 种语言，核心卖点是数据不出本地、无需依赖云端服务。

核心功能：

- **语音克隆与语音设计：**既可以用一段干净的参考录音克隆音色，也可以直接用文字"描述"出一个想要的声音。
- **视频配音与批量制作：**为视频配上时间对齐的语音，并支持故事、有声书等批量任务。
- **听写与转录：**通过悬浮小组件进行语音输入和音频转文字。
- **本地 API 与 MCP 接口：**向 AI Agent 暴露本地能力，并可选接入远程 worker 扩展算力。
- **多引擎与模型管理：**默认搭载 k2-fsa/OmniVoice，也可切换其他引擎，模型可本地安装管理。

适用场景：内容创作者可以用它做视频配音和有声书；开发者和 AI Agent 构建者可以用本地 API/MCP 把语音能力集成进自己的流程；对隐私敏感、不希望语音数据上传云端的用户，以及需要多语言（646 种）支持的国际化场景，都是典型使用者。

技术亮点：采用 Electron 桌面应用形态（早期为 Tauri，现已迁移），安装脚本支持一键部署并保留用户数据；引擎层做了抽象，可插拔切换不同语音模型；同时提供 CLI 式安装、Docker、以及"让 coding agent 自动安装"的 agent 指南，工程化程度较高。AGPL-3.0 授权也体现了其开源立场的坚定。

上手建议：最快的方式是 macOS/Linux 下执行 `curl -fsSL https://voicestudio.sh/install | sh` 一键安装，或从 Releases 下载对应平台包。想贡献代码可从 README 的 "Run the Electron preview from source" 入手，需要 Node.js 22+、Bun、Rust/Cargo 等环境；也可先加入 Discord 社区了解开发方向。

#3

TypeScript

连续 4 天

openrig

Multi-agent harness that runs Claude Code and Codex together as one system

Stars **2,811** Forks **199** Today **+622**

<https://github.com/mvschwarz/openrig>

项目简介：OpenRig 是一个多智能体编排框架（multi-agent harness），它把 Claude Code 和 Codex 这两个独立的 AI 编程助手纳入同一套系统统一管理。它解决的核心问题是：当开发者同时使用多个 AI 编程工具时，会话零散、上下文割裂、缺乏协调，OpenRig 把这些终端会话变成一支有组织、可持久化的"团队"。

核心功能：

- **YAML 定义智能体团队：**用配置文件声明 agent 团队结构（如 owner/checker 角色分工），一条命令即可启动整个团队。
- **跨供应商混合编排：**支持纯 Claude、纯 Codex 或混合编队（如 Claude 主导 + Codex 审查），复用已有订阅账号，无需额外付费。
- **Lead Agent 协调机制：**用户只需与"主导 agent"对话描述目标，由它协调各专家 agent 并汇报需要人工决策的事项。
- **TUI 可视化面板：**以图形和表格形式实时展示各 agent 的运行状态、模型、上下文和座位（seat）详情。

适用场景：主要面向重度使用 AI 编程助手的开发者，尤其是希望在单个代码仓库中长期维护多个 agent 协作、需要角色分工（如开发 + 审查）的团队。适合那些觉得"一堆终端会话难以管理"、希望让 AI 团队持续跟进同一仓库任务的工程师。

技术亮点：采用 TypeScript 编写，基于 tmux 管理持久化会话，用 SQLite 存储状态，体现了"harness 包裹模型、rig 包裹 harness"的分层抽象思想。它巧妙地通过 provider hooks 和权限规则集成，让不同厂商的 CLI 工具在同一内核下协同，而非重新造轮子。

上手建议：从官方 [Getting started 指南](#) 入手，环境需 Node.js 22/24 + tmux（macOS/Linux）。建议先跑 `rig setup --dry-run` 预览改动，再用 `first-project-mixed` 这类起步模板启动一个双 agent 团队体验完整流程。注意：启动会写入 provider hooks 和工作区信任设置，运行前务必阅读"OpenRig 会改动你机器上的什么"并备份相关文件；原生 Windows 暂不支持。

#4

TypeScript

NEW

context-mode

Context window optimization for AI coding agents. Sandboxes tool output (98% reduction), persists session memory, and enforces routing across 17 platforms via MCP + hooks.

Stars **24,366** Forks **1,762** Today **+88**

<https://github.com/mksglu/context-mode>

项目简介：Context Mode 是一个面向 AI 编程代理的 MCP 服务器，核心解决的是“上下文窗口被工具输出快速吃满”这一痛点。它通过沙箱化工具输出、持久化会话记忆和统一路由，让 Claude Code、Cursor 等代理在长时间编码任务中保持上下文干净且连贯。

核心功能：一是工具输出沙箱化，将 Playwright 快照、GitHub issue、访问日志等原始数据压缩高达 98%（如 315 KB 降至 5.4 KB）；二是会话连续性，用 SQLite 记录文件编辑、git 操作、任务和错误，压缩后通过 FTS5 + BM25 只召回相关内容；三是“用代码思考”的提示策略，减少代理的废话输出，从输入和输出两端节省 token；四是跨 17 个平台的 MCP + hooks 路由，统一接入不同 AI 编程环境。

适用场景：主要面向重度使用 AI 编程代理的开发者，尤其是需要长时间、多轮次操作大型代码库或频繁调用外部工具（如浏览器快照、issue 查询、日志分析）的团队。当代理运行 30 分钟后上下文被消耗 40%、压缩后“失忆”时，这个项目能显著改善体验。

技术亮点：用 SQLite + FTS5 做事件索引和 BM25 检索，而不是把历史数据重新塞回上下文，这是“检索代替回灌”的思路；沙箱化工具输出从源头控制 token 膨胀；支持 17 个平台说明抽象层设计得比较通用。Hacker News 570+ 分和 2.4 万星标也侧面印证了痛点真实且方案有效。

上手建议：从 npm 安装 context-mode 并按其文档接入你常用的 MCP 客户端（如 Claude Code、Cursor）开始，先体验沙箱工具输出和会话续接。想贡献的话，可以先看它支持的 17 个平台适配层，补充新平台或优化 FTS5 检索策略都是不错的切入点；注意项目采用 ELv2 许可证，商用前需确认合规。

#5

JavaScript

NEW

ponytail

Makes your AI agent think like the laziest senior dev in the room. The best code is the code you never wrote.

Stars **148,819** Forks **8,004** Today **+675**

<https://github.com/DietrichGebert/ponytail>

项目简介：Ponytail 是一个给 AI 编程助手"注入性格"的技能包，它让 AI Agent 模仿那种扎着马尾、沉默寡言、却总能用一行代码替掉你五十行的资深老开发。它要解决的核心问题是：当前 AI 编程助手倾向于过度设计、堆砌依赖、写大量冗余代码，而 Ponytail 通过约束 Agent 的"表达欲"，让输出更精简、更便宜、更快。项目用 JavaScript 实现，已支持约 20 种 Agent。

核心功能：

- **极简代码生成：**引导 Agent 优先使用平台原生能力（如 `<input type="date">` 而非引入 flatpickr 加包装组件加样式表），从源头砍掉不必要的依赖和样板代码。
- **量化收益：**在真实 Claude Code 会话中编辑 FastAPI + React 开源仓库测得，代码量平均减少约 54%（最高 94%）、成本降约 20%、耗时降约 27%，且安全性保持 100%。
- **安全护栏保留：**与裸的"只写一行"提示词不同，Ponytail 在压缩代码的同时仍保留所有安全检查，避免为了短而牺牲健壮性。
- **广泛兼容：**宣称可与约 20 种主流 AI Agent 协同工作，通过 npm 包 `@dietrichgebert/ponytail` 分发。

适用场景：适合日常使用 AI 编程助手（如 Claude Code 等）进行功能开发的工程师，尤其是那些被 Agent"过度工程化"困扰、希望减少代码审查负担和 token 成本的团队。也适合在已有成熟开源仓库上做增量修改、追求最小 diff 的场景。

技术亮点：最值得关注的是它的**诚实基准测试方法论**——用无头 Claude Code 会话在真实开源仓库上跑 12 个功能工单，以 `git diff` 为评分依据，n=4 取样，并主动修正了早期"单次 80-94%"的夸大宣传，改为按任务区分的均值（54%）与上限（94%）。这种可复现、可证伪的评测态度，在当下充斥着营销话术的 AI 工具中相当难得。此外，它把"懒惰资深开发"这一拟人化隐喻转化为可操作的 Agent 行为约束，是一个巧妙的产品设计。

上手建议：从 npm 安装 `@dietrichgebert/ponytail`，按 README 指引接入你常用的 Agent 即可。想深入理解效果，建议先阅读 `benchmarks/results/2026-06-18-agentic.md` 的完整评测报告，并参考 `examples/` 中的"幸存代码"示例。若要贡献，可从复现基准测试（`benchmarks/` 目录）或补充更多 Agent 兼容性入手，这是项目当前最核心的价值所在。

#6

Python

NEW

MoneyPrinterTurbo

利用 AI 大模型和自动化 workflow，根据主题或关键词一键生成高清短视频。Generate HD short videos from a topic or keyword with an automated AI workflow.

Stars **127,374** Forks **19,921** Today **+464**

<https://github.com/harry0703/MoneyPrinterTurbo>

项目简介：MoneyPrinterTurbo 是一个基于 Python 的开源 AI 短视频生成工具，用户只需输入一个主题或关键词，它就能自动完成从文案撰写到成片输出的全流程。它解决的是短视频内容生产中"创意到成品"链路长、门槛高、耗时多的问题，把原本需要多个工具协作的繁琐流程压缩成一次点击。

核心功能：一是 AI 自动生成视频脚本，调用大模型把关键词扩展成完整文案；二是智能匹配视频素材，根据文案内容自动检索和筛选画面；三是自动生成字幕与背景音乐，并完成音画合成，最终输出高清短视频。此外项目同时提供 WebUI 和 API 两种使用方式，方便普通用户和开发者各取所需。

适用场景：主要面向自媒体创作者、短视频运营者、跨境电商营销人员，以及希望批量生产内容的小团队。典型场景包括日更短视频账号、产品推广素材制作、知识科普内容批量生成等，尤其适合需要"以量取胜"或快速验证选题的运营需求。

技术亮点：项目把 LLM 文案生成、素材检索、TTS 语音、字幕对齐、视频合成等多个环节串成自动化 workflow，并抽象出统一的 API 编排层。它不绑定单一模型厂商，支持 Kimi、火山引擎、MiniMax 等多种大模型和视频生成服务，这种"可插拔"的架构设计让用户能灵活选择性价比方案，也是它获得高关注度的重要原因。

上手建议：普通用户可直接下载 Release 版本，在 Windows/macOS/Linux 上通过 WebUI 快速体验；开发者建议先阅读 README 中的配置说明，准备好大模型 API Key（如 Kimi 或火山引擎）后再运行。想贡献代码的话，可以从 Issues 中标记为 good first issue 的任务入手，重点关注素材匹配质量、多语言支持或新模型接入等方向。

#7

TypeScript

NEW

openclaw

The AI that really does things. Any OS. Any Platform. The lobster way. 🦞

Stars **390,880** Forks **82,208** Today **+136**

<https://github.com/openclaw/openclaw>

项目简介：OpenClaw 是一个开源的本地化 AI 助手，运行在用户自己的电脑上，并通过 Discord、iMessage、Slack、Telegram、WhatsApp 等 20 多个聊天渠道与用户交互。它解决的核心问题是：让用户拥有一个数据自主可控、可跨平台、可自由替换模型后端的个人 AI 助理，而非绑定某个云端服务。

核心功能：

- **多渠道路由：**把同一个助手接入 WhatsApp、Telegram、Slack、Discord、Signal、iMessage 等主流聊天平台，用户在自己习惯的应用里就能调用。
- **Gateway 本地控制平面：**统一管理会话、工具、事件与渠道连接，通过 Control UI、CLI 或 TUI 接入，个人笔记本和团队部署只差配置。
- **插件化模型与 Agent 框架：**Claude、Codex、本地模型等都可作为插件热插拔，无需改动其他部分。
- **跨平台原生应用：**提供 macOS、iOS、Android、Windows、Linux 客户端，并支持语音、Canvas、摄像头、屏幕等设备本地能力。

适用场景：适合注重隐私与数据主权的个人用户，把 AI 助手跑在自己的硬件上并接入日常聊天工具；也适合需要共享部署的小团队，用同一套 Gateway 做团队级助理。对希望自由切换模型供应商、避免被单一云厂商锁定的开发者尤其有吸引力。

技术亮点：采用 TypeScript 编写，架构上强调"可信网关 + 不可信执行 + 确定性策略"的分离设计，这是其安全模型的核心。默认几乎不"回传"数据，仅做每日版本检查，匿名统计需主动开启，且可通过 `update.checkOnStart: false` 一并关闭。项目由独立的 501(c)(3) 基金会治理，无付费层级、无托管服务、无代币，商业模式上的克制本身就是一种技术信任背书。

上手建议：最快路径是执行官方一键安装脚本（macOS/Linux 用 `curl ... | bash`，Windows 用 PowerShell），安装器会自动配置 Node 运行时并启动引导向导。熟悉 Node 的用户可直接 `npm install -g openclaw@latest`，然后运行 `openclaw onboard --install-daemon`，再用 `openclaw gateway status` 和 `openclaw dashboard` 验证。想深入贡献的话，建议先读官方文档中的 "Why OpenClaw" 架构说明和 VISION.md，理解其网关/执行分离的设计哲学，再从渠道适配或工具/技能插件入手。

#8

Python

NEW

awesome-claude-skills

A curated list of awesome Claude Skills, resources, and tools for customizing Claude AI workflows

Stars **76,020** Forks **8,870** Today **+118**

<https://github.com/ComposioHQ/awesome-claude-skills>

项目简介：这是一个由 ComposioHQ 维护的 Claude Skills 精选资源列表，汇集了 1000+ 生产可用的技能包（Skills）与插件。它解决的核心问题是：让 AI Agent 不只是“会聊天”，而是能按照可复用的标准化 workflow 真正执行具体任务，并连接外部应用完成实际动作。

核心功能：

- **技能目录分类：**按文档处理、开发工具、数据分析、商业营销、创意媒体等 10 大类整理技能，方便按场景检索。
- **connect-apps 插件：**通过一条命令让 Claude 具备发邮件、建 issue、发 Slack 等真实操作能力，背后由 Composio 的 MCP Gateway 提供 1000+ 应用集成与鉴权。
- **渐进式加载机制：**技能启动时只加载名称和描述（约 100 token），按需才载入完整内容，让单个 Agent 可承载数百个技能而不撑爆上下文。
- **开放标准支持：**兼容 Claude Code、Claude.ai、OpenAI Codex、Cursor、Gemini CLI、Windsurf 等多个 Agent 平台。

适用场景：适合希望把 Claude 或其他编码 Agent 从“文本生成器”升级为“任务执行器”的开发者、产品团队和自动化爱好者。典型场景包括批量文档处理、代码工作流自动化、跨应用（邮箱/工单/Slack）操作编排，以及企业内部沉淀可复用的 AI 工作规范。

技术亮点：技能以 SKILL.md（YAML frontmatter + Markdown 指令）为最小单元，可附带 scripts/ 与 references/ 按需加载，本质是“工作流定义层”，与 MCP（连接层）、Tools（函数层）形成清晰分工。这种渐进式加载设计是让 Agent 承载大量技能而不失控的关键工程取舍。

上手建议：最快路径是按其 Quickstart 安装 connect-apps 插件，用 /connect-apps:setup 配置 API key 后体验真实动作；想深入则阅读 "What Are Claude Skills?" 与 "Creating Skills" 章节，参照 SKILL.md 格式写自己的第一个技能。贡献方面，项目欢迎 PR，可先提交符合分类规范的技能条目或补充资源链接。

#9

Shell

NEW

skills

Skills for Real Engineers. Straight from my .agents directory.

Stars **272,754** Forks **22,946** Today **+736**

<https://github.com/mattpocock/skills>

项目简介

这是知名 TypeScript 教育者 Matt Pocock 开源的一套「AI 编码 Agent 技能包」，直接从他日常使用的 .agents 目录中提取而来。它要解决的核心问题是：当前 AI 编码助手（Claude Code、Codex 等）经常「不懂你要什么」就开始写代码，导致返工和失控，而现有方案（GSD、BMAD、Spec-Kit）又过度接管流程、剥夺开发者控制权。

核心功能

- **/grill-me 与 /grill-with-docs**：在动手前让 Agent 反过来「拷问」你，通过密集提问澄清需求，这是作者最受欢迎的技能，直击「Agent 没做我想要的」这一头号失败模式。
- **/setup-matt-pocock-skills**：每个仓库运行一次的初始化命令，配置 issue tracker（GitHub / Linear / 本地文件）、triage 标签和文档存放位置。
- **/triage**：配合标签体系对工单进行分诊管理。
- **统一的领域语言约定**：借鉴 DDD 思想，压缩 Agent 过度啰嗦的输出，让对话和代码共享同一套术语。

适用场景

适合**认真做工程的开发者**，而非「vibe coding」式的随意尝试——作者明确强调这些技能用于真实项目开发。典型场景包括：用 Claude Code 或 Codex 开发中大型应用时，需要在编码前对齐需求、管理工单、控制 Agent 输出质量。任何觉得 AI 助手「自作主张」或「话太多」的工程师都能受益。

技术亮点

- **小而可组合**：刻意不做大而全的框架，每个技能独立、易改、可混搭，且**不绑定特定模型**。
- **双安装哲学**：Claude Code 插件走「订阅式」只读托管、自动更新；skills.sh 走「复制式」把可编辑文件写进你的仓库，二选一避免重复。

- **工程经验驱动：**每个技能都对应一个具体失败模式，并引用《程序员修炼之道》《领域驱动设计》等经典，理论根基扎实。

上手建议

最快路径是 Claude Code 用户直接 `claude plugins install mattpocock-skills`，其他 Agent 用 `npx skills@latest add mattpocock/skills`（务必勾选 `setup-matt-pocock-skills`），然后在仓库里运行一次 `/setup-matt-pocock-skills` 完成配置。想深入贡献的话，建议先读 `.agents/adr/` 下的架构决策记录（如 0002 号关于以 Claude Code 插件形式发布的说明），理解设计取舍后再动手改造。

#10

TypeScript

NEW

hyperframes

Write HTML. Render video. Built for agents.

Stars **54,527** Forks **4,955** Today **+352**

<https://github.com/heygen-com/hyperframes>

项目简介：HyperFrames 是 HeyGen 开源的一个框架，能把 HTML、CSS、媒体资源和可定位（seekable）动画转换成确定性的 MP4 视频。它解决的核心问题是：让视频渲染变得像写网页一样简单、可复现，并且专门为 AI 编码代理（agents）设计，使 AI 能直接"写 HTML 出视频"。

核心功能：一是 HTML/CSS 到 MP4 的确定性渲染流水线（CLI 本地可用）；二是面向 AI 编码代理的 21 个 skills，通过 /hyperframes 路由器按需加载 workflow；三是完整的生产闭环——规划视频、写 HTML、接动画、加媒体、lint、预览、渲染；四是可作为托管式创作 workflow 背后的渲染核心；五是提供 Playground、Catalog 组件库和 Showcase 等配套资源。

适用场景：适合需要批量、程序化生成视频的开发者与团队，比如产品介绍、数据图表动画、营销短视频等；也特别适合用 Claude Code、Cursor、Gemini CLI 等 AI 编码代理来"一句话生成视频"的创作者。HeyGen 自身的产品场景（数字人/营销视频）也是典型用例。

技术亮点：最值得关注的是"确定性"渲染——同样的 HTML 输入产出同样的 MP4，这对自动化流水线和 AI 生成至关重要；其次是"seekable animations"设计，让动画可被精确定位到任意时间点，便于帧级渲染；再者是把 skills 体系深度集成进 agent 生态，用路由器按需加载而非一次性灌入全部能力，工程上相当克制。项目要求 Node.js ≥ 22 ，采用 Apache 2.0 许可。

上手建议：最快路径是用 AI 代理——在 Claude Code 里装插件（`claude plugin marketplace add heygen-com/hyperframes`），或运行 `npx skills add heygen-com/hyperframes` 装核心 skills，然后直接给代理下 prompt 试生成一段 10 秒视频。想深入的话，从官方 Quickstart 和 Playground 入手，读 /hyperframes 这个路由器 skill 理解整体能力地图。贡献方面可先看 Catalog 里的 block 组件和 skills 目录结构，Discord 是主要交流渠道。注意 skills add 的注册表可能滞后于 main，需要最新版时用 `npx hyperframes skills update`。

数据来源: github.com/trending

报告日期: 2026-09-30

由 DeepSeek AI 提供智能分析 | 自动生成